

c# Programmieren (c) HTL-Leonding

BattleShip - Schiffe versenken

Lehrziele

- mehrdimensionale Arrays
- Datei Lesen/Schreiben
- Board

Aufgabenstellung

Gesucht ist ein Programm für das Spiel **Schiffe versenken**.

In dieser Varianten soll "nur" das Erstellen eines Spielfelds unterstützt werden.

Der Anwender kann durch eine Konsolen-Eingabe Schiffe am Spielfeld setzen (oder auch wieder löschen). Das Spielfeld soll in einer Datei gespeichert und aus einer Datei wieder geladen werden können.

Programmablauf

Schreiben Sie ein Programm **BattleShip** mit folgenden Eigenschaften:

- Beim Programmstart wird die Dimension des Spielfelds eingegeben.
Da es sich um ein quadratisches Spielfeld handelt, muss nur eine Dimension eingegeben werden, z.B.:
15.
Erlaubte Größen sind 10x10, 15x15 oder 20x20.
- Anschließend wird das **Board** mit den eingegebenen Dimensionen erstellt.
- Der Benutzer wird aufgefordert, die folgenden Kommandos einzugeben:
 - **exit**
Das Programm wird sofort (ohne Rückfrage) beendet.
 - **load filename**
Mit dieser Eingabe wird das aktuelle Spielfeld zurückgesetzt und die Schiffe aus der angegebenen Datei geladen.
Nur gültige Spielfelder werden geladen. Sind die Positionen der Schiffe ungültig, wird eine Meldung ausgegeben und das Programm verlassen.
Beispiel: *load mygame.csv*
 - **save filename**
Das aktuelle Spielfeld wird in die angegebene Datei gespeichert.
Eine eventuell vorhandene Datei wird NICHT überschrieben sondern auf *bak* umbenannt.
 - **row,col**
Beispiel: mit 3,5 wird an die Stelle 3,5 (row,col) ein neues Schiff positioniert.
Im Board wird die Position mit 'o' belegt.
Eine Fehlermeldung ist auszugeben, wenn das Schiff nicht gesetzt werden kann:
 - Wenn im Feld bereits ein Schiff an dieser Position vorhanden ist.
 - In den Nachbarfeldern (horizontal, vertikal und diagonal) ein Schiff existiert.
Ein Schiff darf an eine Randposition gesetzt werden. Die Nachbarpositionen, die ausserhalb des Feldes liegen, werden ignoriert.

- **!row,col**

Mit dieser Eingabe wird ein Schiff gelöscht. Eine Fehlermeldung ist auszugeben, wenn an der Position kein Schiff gespeichert ist.

Programmdesign

Achten Sie bei der Umsetzung auf ein sauberes Design Ihres Programms.

In dem bereitgestellten Programm-Template sind Unittests vorhanden. Implementieren Sie die Methoden so, dass **alle** Unittests erfolgreich ausgeführt werden können. Änderungen an den *Unittests* sind nicht erlaubt.

Damit die Unittests ausgeführt werden können, müssen sie folgende (Hilfs-)Methoden umsetzen:

- `bool CanSetShip(int[,] battlefield, int row, int col)`
Überprüft, ob ein Schiff an die angegeben Position gesetzt werden kann.
Die Position muss leer sein und es dürfen sich keine Schiffe in der *Nachbarschaft* befinden.
- `bool GetShipCount(bool[,] battlefield)`
Zählt alle am Spielfeld platzierte Schiffe.
- `void SaveField(bool[,] battlefield, string fileName)`
Das Spielfeld (gespeichert im `battlefield`) wird in eine Datei geschrieben. Die CSV Datei hat folgende Spalten: "No;Row;Col" (No ist eine fortlaufende Nummer).
Hinweis: es werden nur *Schiffe* in die Datei geschrieben, die leeren Position jedoch nicht.
- `int[,]? LoadGame(int boardSize, string fileName)`
Ein Spiel wird aus der angegebenen Datei geladen. Das Format der Datei entspricht der CSV Datei der Methode `SaveGame`.
Hinweis: Die Datei kann eine "ungültige" Definition enthalten (ungültige Positionen der Schiffe). In diesem Fall liefert die Methode `null` als Rückgabewert.

Testen Sie das Programm ausführlich.

Bildschirmausgabe

```
BattleShip
=====
Board size [10,15 or 20]: x
Ungültige Eingabe
Board size [10,15 or 20]: 19
Ungültige Größe
Board size [10,15 or 20]: 10
"row,col" to set a ship, e.g. 5,7
"!row,col" to clear a ship, e.g. !5,7
"exit" to quit program
"save filename" to save the field (and continue)
"load filename" revert and load the field from a file.
=>
```